Data: collection of objects & attributes

Types: i. Qualitative (categorical)
- Gender, color

ii. Quantitative (numeric)
- Height, weight.

Data set: collection of objects & attributes that are organised together for analysis.

-> Record-based Data set: (SMART-G)

i. Sparse Data matrix: Mostly zero values, only few imp

Ex: $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$

ii. Data Matrix: obj - rows
Att - cols

Ex:

Stu	Marks
A	90
B	95

iii. Record data: each obj has same attributes. Ex: Student DB.

iv. Transaction Data: records contain set of items

Ex: customer purchase: Milk, Tea

v. Graph data: obj - nodes
relations - edges.

Ex: (A) $\xrightarrow{\text{friend}}$ (B)

-> order-based:
Data arranged in order in time or space.

Ex: Stock prices.

Types: S^2T: Sequential, spatial, Time series

-> Non-record data:
- images
- text
- Audio
- Graph

} handled using NLP, Graph modeling

-> Characteristics of Data sets

i. Dimensionality: no. of attributes

ii. Sparsity: contains many no. of zero values.

iii. Resolution: level of detail

iv. Size: large -> big storage & processing

(DRS^2)

UNIT-1 | Attributes: property or characteristic of an object.

Ex: obj-student, attributes: name, age

Attribute value: value assigned to an attribute.

Properties: D - Distinctness, R - Meaningful ratio
D - Meaningful difference, O - order.

Measurment: process of assigning values to attributes.

Types of Attributes: (NOIR)

1. Nominal: only names/ categories, no order
Ex: Blood group.
2. Ordinal: ordered categories. Ex: Rank, Grade
3. Interval: numeric with meaningfull difference, no true zero Ex: Temp (°c)
4. Ratio: numeric with true zero, Ex:- Height, weight

Attributes by no. of values:

-> Binary attribute: contains 0/1 value.
Ex: Yes/No

i. Symmetric: Both values are equally imp
Ex: Male/ Female

ii. Asymmetric: one value is more imp than other
Ex: Disease (+ve).

Numeric Attributes:

i. Discrete: countable values Ex: no. of students

ii. Continuous: non-countable values. Ex: temp.

Data quality: Ensures correctness & reliability for analysis

Factors: (ACCT)
- Accuracy: correct values
- completness: no missing data
- consistency: no contradictions
- Timeliness: updated data

Measurement & Data collection issues: (NMIDS)

i. Noise: random error in data.

ii. Missing values: unavailable data.

iii. Inconsistance format: incorrect values.

iv. Duplicate Data: repeated data.

v. Sensor errors: error or mistake in sensors.

-> precision: Repeatability of measurement.

-> Accuracy: closeness to true value.

-> Bias: systematic error.

Basic Data types in R:

Datatype: the type of data stored.

X ← 10 #numeric

Y ← 10L # integer

Z ← TRUE # logical

a ← "R" # character

b ← 2+3i # complex

class(x) ⇒ numeric

used to check datatype

coercion: Automatic conversion of data-type

Ex: v ← c(1, "A", 2L)

character

Arrays: Multi-dimensional data structure that stores elements of same type.

Syntax: array(data, dim, dimnames)

Example:

arr ← array(1:6, dim=c(2,3))

Class: blueprint that defines how objs are treated.

Ex:

person ← list (name="Ravi", age=20)

class(person) ← "person"

S3 class

Dataframe: table-like structure which has elements of different type (columns)

syntax: df_name ← data.frame(data)

Ex: df ← data.frame(

id = c(1,2,3)

name = c("A", "B", "C"),

age = c(20, 21, 22))

Subsetting: df[1,2]

df[["age"]] df$name

UNIT-2

vectors: A vector is a collection of same type of elements.

Syntax: vec_name ← c(value1, value2 ...)

Example: v1 ← c(1,2,3) length(v1) # 3

Arithmetic operations:

v1 ← c(10,20,30)

v2 ← c(1,2,3)

v1+v2 # 11 22 33

v1-v2 # 9 18 27

v1*v2 # 10 40 90

v1/v2 # 10 10 10

* Dot product sum(v1 * v2)

Vector subsetting:

Extracting elements from vector:

v ← c(10,20,30,40)

v[1] # 10

v[1:3] # 10 20 30

Matrices: 2D structure with same type elements.

Syntax: mat_name ← matrix(data, nrow, ncol)

Example: m ← matrix(1:6, nrow=2, ncol=3)

Naming:

m ← matrix(1:4, nrow=2

dimnames = list(c("r1", "r2"), c("c1", "c2")))

Matrix subsetting: m ← matrix(1:6, nrow=2) # 1 3 5 / 2 4 6

m[1,2] # 3

m[1,] # 1 3 5

m[, 2] # 3 4

Factor: used to store categorical data.

Syntax: factor_name ← factor(vector)

Example: f ← factor(c("A", "B", "A")) # A B A

levels(f) # "A" "B" #Levels: A B

summary(f) # A B

2 1

modifying levels:

levels(f) ← c("X", "Y")

f # X Y X

#Levels: X Y

ordered factors: size ← factor(c("S", "M", "L"), ordered = TRUE, levels=c("S", "M", "L"))

comparing: s1 ← factor("L", levels=c("S", "M", "L"), ordered = TRUE) # S L M

s2 ← factor("M", levels=c("S", "M", "L"), ordered = TRUE) # Levels: S < M < L

s1 > s2 # TRUE

rbind: used to add row

cbind: used to add column

sort(df$age)

df ← cbind(df, weight=c(60,65,70))

df ← rbind(df, "D", age=24, 72)

UNIT-3

List: collection of diff data types.
- Heterogeneous data

Syntax: list_name <- list(ele1, ele2 ...)

Example:
```
my-list <- list("Hi", 21, c(1,2,3))
print(my-list[1])
```

```
my_list[3] <- c(4,5,6)
my_list[3] <- NULL
```
} List Manipulation

Merging lists:
```
m_list <- list(list1, list2)
```

EX:
```
list1 <- list(1,2,3)
list2 <- list("a","b")
merged <- list(list1, list2)
```

List to vector:
```
unlist(list_name)
```

EX:-
```
list1 <- list(1,2,3)
v <- unlist(list1)
print v
```

Iteratives:

i. while loop:
```
i <- 1
while (i<=5){
    print(i)
    i <- i+1 }
```

ii. For loop:
```
for (i in 1:5) {
    print(i) }
```

looping over list:
```
my-list <- list(1,2,3)
for(i in my-list) { print(i) }
```

Looping over vector:
```
v <- c(10,20,30)
for(i in v) { print(i) }
```

Recursion: function calling itself.
```
fact <- function(n){
    if(n==1) {
        return(1) }
    else { return (n* fact(n-1))
    }
}
print(fact(5)) #120
```

Loading package:
```
install.packages("ggplot2")
library(ggplot2)
```

Math functions:
abs(), sqrt()
log(), log10(),
exp(), floor(),
sin(), cos(), tan()

Relational operators:
== != > < >= <=

Ex: print(5<3)
#FALSE

with vectors:
```
v <- c(10,5,15,8)
print(v>10)
# FALSE FALSE TRUE FALSE
```

Logical operators:
& | !
```
x <- 10
print(x>5 & x<15)
print(x>20 | x<15)
print(!(x>5))
```

vectors with logical operators:-
```
v1 <- c(TRUE, FALSE)
v2 <- c(FALSE, FALSE)
print(v1 & v2)
```

conditional statement:

->if: syntax: if (cond) { ... }
```
Ex: x <- 5
    if(x>0) {
        print("true") }
```

->if-else
```
if (cond){
    ... }
else { ... }
```

-> if-elseif ladder
```
if (cond1) { ...
} else if (cond2 {
    ... } else { }
```

Functions: It is reusable block of code.

syntax: func-name <- function (parameters) {
```
    ... }
```

Example:
```
Square <- function (x) {
    return (x*x) }
print (square(5))
```

Nested function: function inside function

Scope: Accessability of variables.
i. Global ii. local

```
x <- 10 #Global
func <- function() {
    x <- 5
    print(x) #5
}
func()
print (x) #10
```